

实验一 Linux 下编译、运行和调试 C/C++程序

1. 实验目的

掌握 Linux 下程序的编辑、编译、运行及调试方法，并能熟练应用。

2. vi 编辑器

(1) vi 编辑器的简介

vi 是 Visual interface 的简称，它可以执行输出、删除、查找、替换、块操作等众多文本操作，用户可以根据自己的需要对它进行定制。vi 不是一个排版程序，它不像 Word 那样可以对文字、格式、段落等其他属性进行编排，它只是一个全屏幕文本编辑程序，它没有菜单，只有命令。

(2) vi 的 2 种工作模式

① 命令模式：在命令模式中，键入的任何内容都被解释成命令。具体命令的含义可参照附录II。

② 输入模式：在输入模式中，键入的任何内容都直接插入到编辑缓冲区中。在进入 vi 编辑器时，vi 处于命令模式，键入 a、i、o 都可进入输入模式，具体输入位置不同，可参照附录查看具体含义。

(3) vi 的启动、编辑和退出

① vi 的启动：vi [file]，如果文件存在，则打开该文件，如果文件不存在，则创建一个新文件；如果省略文件名，则创建一个无名的新文件，在存盘时需指定文件名。

② vi 的编辑：进入 vi 后，vi 处于命令模式下，键入 a、i、o 即可进入文本输入模式，此后输入的字符将显示在编辑屏幕上。键入 Esc，即可离开输入模式，回到命令模式。

③ vi 的退出：在回到命令模式后，键入“:”，出现“:”提示符后可键入以下字符，具体含义如表 2.1 所示。

表 2.1 vi 基本操作命令

: w	保存
: w [file]	另存为 file
: wq	保存退出
: wq [file]	另存为 file 后退出
: q!	不保存退出
: x	保存退出，同: wq

3. 程序的编译和运行

通过一个简单的 hello 程序，展示整个编译运行过程。

(1) vi hello.c 进入 vi，新文件名为 hello

(2) 键入 a 进入编辑模式，键入以下程序

```
#include<stdio.h>
void main( )
{
    printf("hello!\n");
}
```

(3) 编译 hello 程序

```
gcc hello.c (编译 hello.c, 默认情况下生成 a.out)
gcc -o hello hello.c (生成名为 hello 的可执行文件)
```

(4) 运行 hello 程序

```
./hello
```

屏幕显示运行结果: hello!

4. 程序的调试

□ gdb 的介绍

gdb 是 GNU 开源组织发布的一个强大的 Unix 下的程序调试工具。一般来说, **gdb** 主要帮助你完成下面四个方面的功能:

- ① 启动你的程序, 可以按照你的自定义随心所欲的运行程序。
- ② 可让被调试的程序在你所指定调试的[断点](#)处停住 (断点可以是条件表达式)。
- ③ 当程序被停住时, 可以检查此时你的程序中所发生的事。
- ④ 动态改变程序的执行环境。

(2) gdb 的使用

必须要把调试信息加到可执行文件中, 才可以使用 **gdb** 调试程序。在编译时加上 **-g**, 如 **hello.c** 程序, 在编译时应输入 `gcc -g hello.c -o hello`。然后输入 `gdb hello` 即可启动 **gdb** 调试 **hello** 程序。

(3) gdb 调试程序示例

一个有错误的 C 源程序 **bugging.c** 代码如下:

```
1  #include <stdio.h>
2  static char buff [256];
3  static char* string;
4  void main () {
5      printf ("Please input a string: ");
6      fgets (string, 10, stdin);
7      printf ("\nYour string is: %s\n", string);
8  }
```

上面这个程序非常简单, 其目的是接受用户的输入, 然后将用户的输入打印出来。该程序使用了一个未经过初始化的字符串地址 **string**, 因此, 编译并运行之后, 将出现 **Segment Fault** 错误:

```
$ gcc -o bugging -g bugging.c
```

```
$ ./bugging
```

```
Please input a string: abcde
```

```
Segmentation fault (core dumped)
```

为了查找该程序中出现的問題, 利用 **gdb**, 并按如下的步骤进行:

- ① 运行 `gdb bugging` 命令, 装入 **bugging** 可执行文件;
- ② 执行 `set debug-file-directory = /usr/lib/debug` 命令, 将 **GDB** 搜索单独调试信息文件的目录设置为全局调试信息目录;
- ③ 使用 `run` 命令运行已装入的可执行程序 **bugging**;
- ④ 使用 `where` 命令查看程序出错的地方;
- ⑤ 利用 `list` 命令查看调用 `gets` 函数附近的代码;
- ⑥ 唯一能够导致 `gets` 函数出错的因素就是变量 **string**, 用 `print` 命令查看 **string** 的值;
- ⑦ 在 **gdb** 中, 可以直接修改变量的值, 只要将 **string** 取一个合法的指针值就可以了,

为此，在第 6 行处设置断点 `break 6`;

⑦ 程序重新运行（使用 `run` 命令）到第 6 行处停止，这时，可以用 `set variable` 命令修改 `string` 的取值，`set string = buff`;

⑧ 然后用 `continue` 命令继续运行，将看到正确的程序运行结果;

⑨ 使用 `quit` 推出 `gdb`。